

조운호 PLATFORM ENGINEER

배포 흐름을 만들고, 운영까지 설계합니다.

Git 기반 Preview 배포와
Kubernetes·Proxmox 운영 도구를 구현합니다.

DEPLOYMENT TRACE

HM-01



● VERIFIED PATH

READY

실행한 뒤, 실제 상태를 다시 확인합니다.

Git commit을 Docker Preview로 배포하고, 자연어 명령을 Kubernetes 작업으
로 연결하며, Proxmox VM 변경 결과를 다시 확인하는 도구를 만들었습니다.

01	Heimdall PREVIEW DEPLOYMENT	Exact commit을 빌드하고 health와 route를 확인한 뒤 Preview를 전환합니다.	DEPLOY
02	K-Le-PaaS KUBERNETES OPERATIONS	자연어 요청을 Kubernetes 조회·실행 API와 외부 URL 흐름으로 연결했습니다.	OPERATE
03	Gjallar PROXMOX OPERATIONS	API 작업은 task와 실제 상태를, guided 작업은 after-state를 다시 확인합니다.	VERIFY

VALIDATION ENVIRONMENT

Heimdall은 홈랩 배포로, Gjallar 최신 구조는 자동 테스트와 production build로 검증했습니다.

Proxmox VE 3-node

IPFire segmentation

NAS / NFS

WireGuard / OCI proxy

하나의 질문에서, 두 개의 책임으로.

K-Le-PaaS에서 운영 흐름을 연결하며 남은 ‘실행 뒤 무엇을 성공이라 할 것인가’라는 질문을, 개인 프로젝트에서는 배포와 Proxmox 운영이라는 서로 다른 책임으로 나눠 다뤘습니다.

TEAM PROJECT / 2025.09-12	K-LE-PAAS	QUESTION	DEPLOYMENT CONTROL	HEIMDALL
			검증된 Preview만 활성화합니다.	commit과 설정을 고정하고 candidate health와 적용된 route를 probe한 뒤 active Preview를 확정합니다.
CODE HISTORY	현재 두 제품은 서로 다른 코드베이스와 제품 경계를 가집니다.	요청을 실행한 뒤, 어떤 근거로 성공을 판단할 것인가?	INFRASTRUCTURE OPERATIONS	GJALLAR
			VM 변경 뒤 실제 상태를 확인합니다.	API 작업과 guided 작업을 각기 다른 실행·사후 확인 절차로 마무리합니다.

Gjallar는 초기 Heimdall snapshot에서 Proxmox 운영으로 재정의됐고, 현재 Heimdall Python은 이후 별도로 재구축됐습니다.

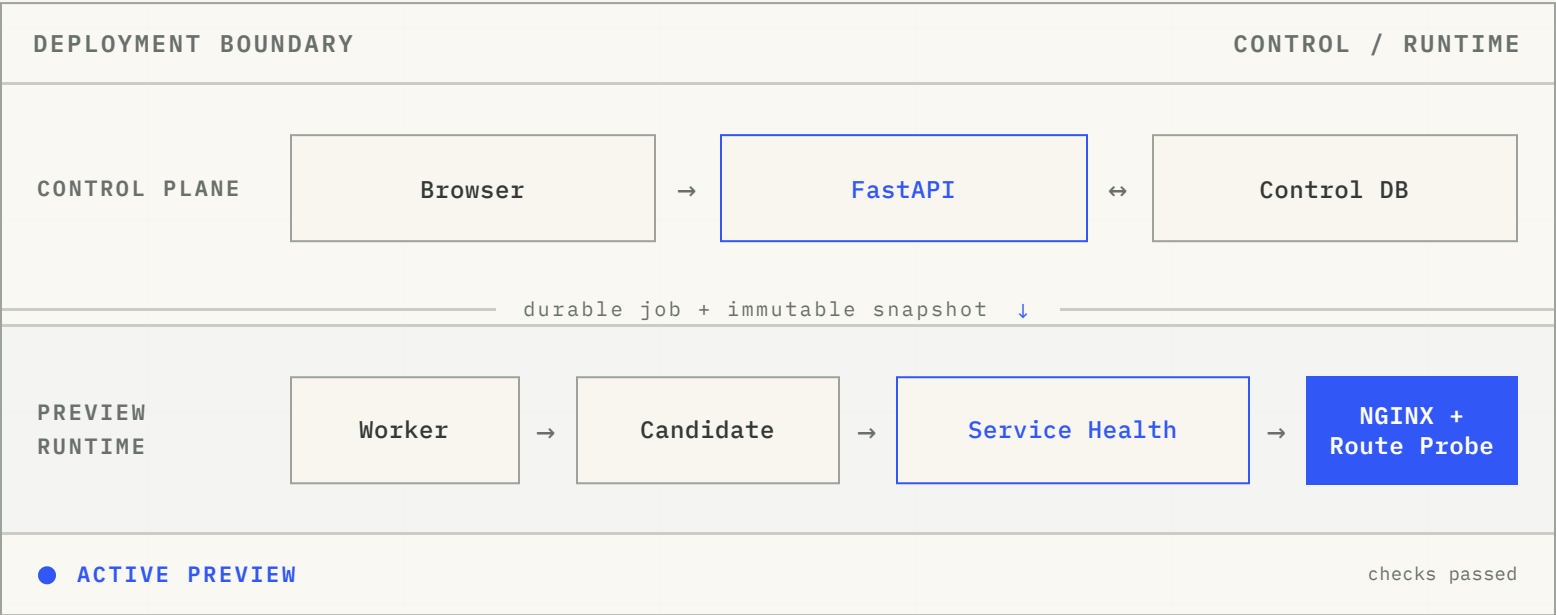
HEIMDALL

Git commit에서 안정적인 Preview까지.

공개 GitHub 저장소의 `main` commit을 격리된 Docker candidate로 빌드
하고, health 확인과 적용된 route의 probe를 통과한 경우에만 active
Preview로 확정합니다.

CORE PRINCIPLE

route를 적용한 뒤 실제 응답까지 확인해야
active Preview로 확정합니다.



CURRENT SCOPE

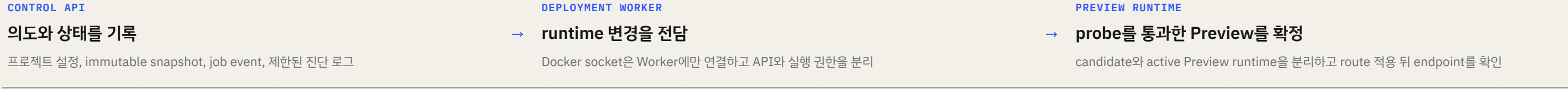
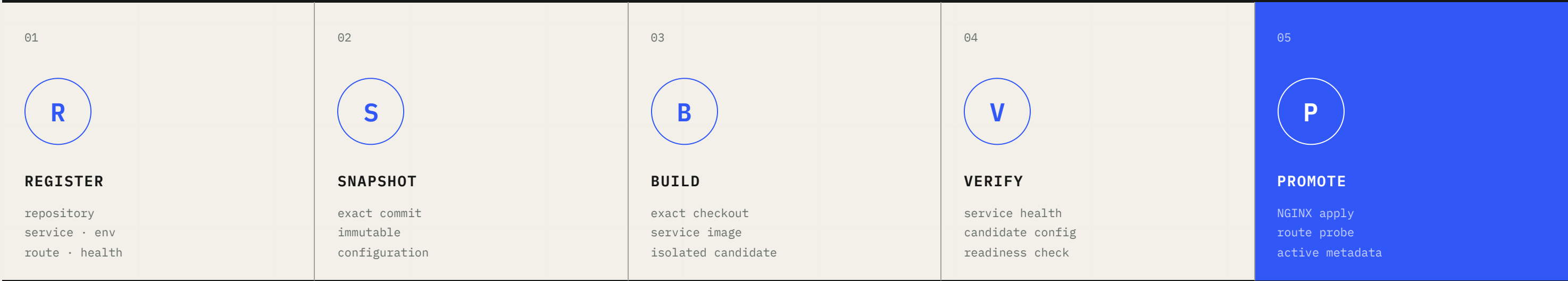
Public GitHub · fixed main · manual deployment · single Docker host

INPUT PUBLIC GITHUB / FIXED MAIN	EXECUTOR WORKER OWNS DOCKER SOCKET	OUTPUT LOOPBACK PREVIEW ENDPOINT	BOUNDARY MANUAL / SINGLE-HOST ALPHA
ROLE 설계 / FastAPI 백엔드 / 배포 Worker / React UI	STACK Git · Docker · NGINX · FastAPI · React · PostgreSQL		

VERIFIED PROMOTION

동작을 확인한 candidate만 Preview가 됩니다.

배포 대상을 먼저 고정하고, 기존 Preview와 분리된 환경에서 candidate service 를 확인합니다. NGINX route 적용 뒤 실제 endpoint 응답까지 확인해야 active metadata가 확정됩니다.



IMPLEMENTATION NOTE Control Plane Compose와 프로젝트별 Preview runtime은 수명 주기와 권한이 분리되어 있습니다.

FAILURE IS A STATE

candidate 실패 뒤 기존 Preview 경로의 복구를 시도합니다.

01 / ROUTE CHECK FAILURE

실패한 route를 active로 확정하지 않습니다.

NGINX route를 적용한 뒤 probe가 실패하면 active Preview metadata를 확정하지 않고 기존 정상 route의 복구를 시도합니다.

02 / INTEGRATION EVIDENCE

두 번째 배포가 실패해도 이전 응답이 남습니다.

실패한 재배포 뒤에도 기존 deployment ID, host port, HTTP response가 유지되는 경로를 통합 테스트로 확인했습니다.

03 / UNCERTAIN RUNTIME

모르는 상태를 성공으로 바꾸지 않습니다.

runtime 상태가 불명확하면 자동 성공이나 즉시 삭제로 단정하지 않고 reconciliation이 필요한 상태로 보존합니다.

CURRENT PRODUCT BOUNDARY

ALPHA / SINGLE-HOST

SUPPORTED NOW

- Public GitHub repository
- Manual deployment
- Loopback Preview endpoint
- Fixed `main` branch
- Docker candidate + NGINX route
- External managed PostgreSQL provisioning

NOT INCLUDED YET

- ☐ Private Git / webhook / auto deploy
- ☐ Multi-host / multi-user
- ☐ Public domain / TLS
- ☐ Image or data rollback

IMPLEMENTATION REFERENCES

NGINX gateway tests ↗ Worker runtime smoke ↗ Product scope ↗

K-LE-PAAS

여러 운영 도구를 하나의 흐름으로.

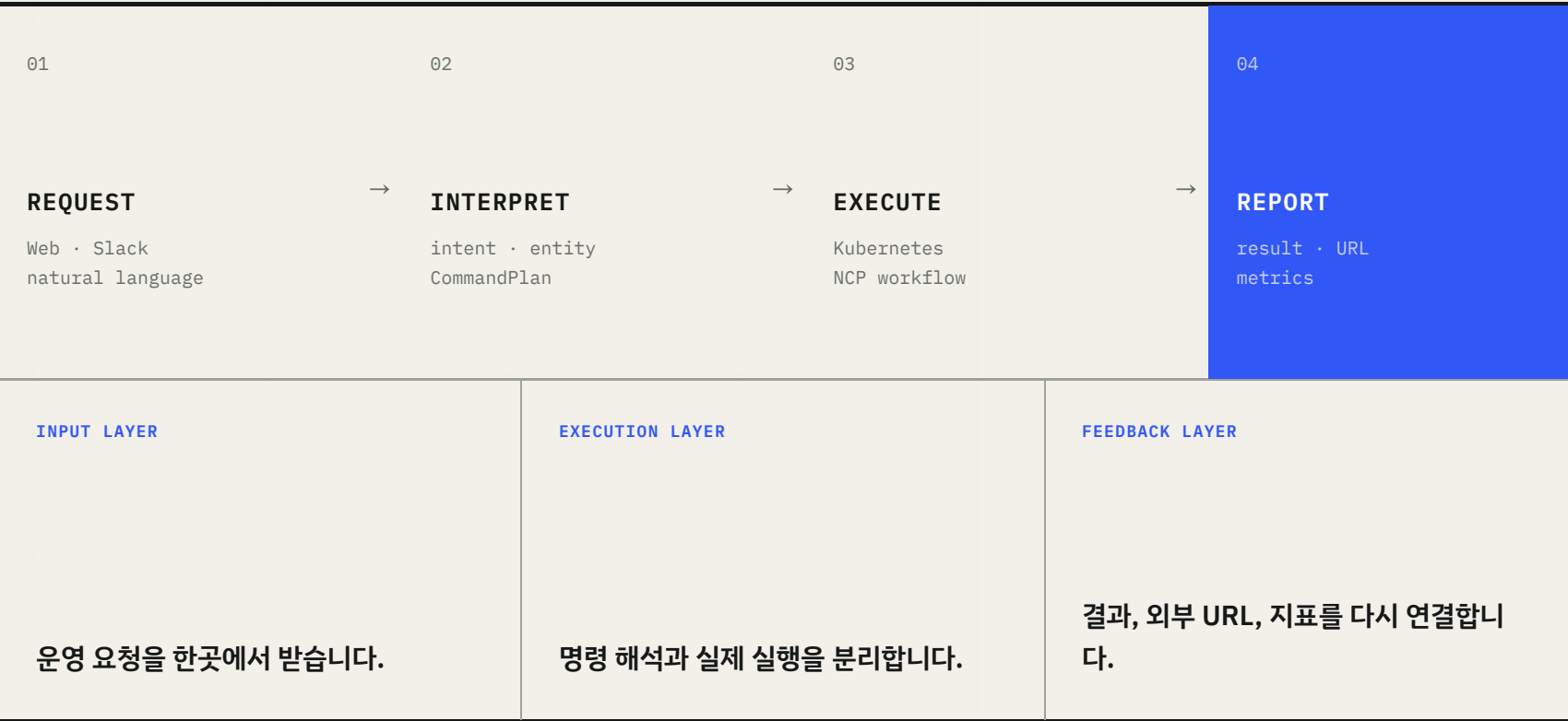
웹과 Slack의 자연어 요청을 Kubernetes와 NCP 운영 흐름에 연결하고, 실행 결과와 접근 정보를 다시 전달하는 팀 기반 클라우드 운영 플랫폼입니다.

MY CONTRIBUTION

NLP commands / Kubernetes API /
Deployment URL / NKS Monitoring

CONTEXT 한국인공지능·소프트웨어산업협회 Cloud 과정 팀 프로젝트

STACK Kubernetes · Gemini · FastAPI · Prometheus · WebSocket · NCP



MY CONTRIBUTION

자연어 요청을 실행·조회 API와 외부 URL까지 연결했습니다.

01 **NLP + KUBERNETES OPERATIONS**

Gemini가 해석한 요청을 intent와 entity로 구조화하고 상태·로그·외부 URL·재시작·스케일링·버전 롤백·리소스 조회 함수와 연결했습니다.

02 **DEPLOYMENT ACCESS**

Ingress 외부 주소와 NCP SourceDeploy 흐름의 사용자·저장소별 service URL을 생성하고 저장·조회하는 경로를 구현했습니다.

03 **NKS MONITORING**

Prometheus의 CPU·memory·disk·network 지표를 API와 WebSocket endpoint로 제공했습니다.

TEAM SCOPE 웹 콘솔, 인증, 전체 배포 pipeline, Slack 연동은 팀이 함께 구성했습니다.

K-LE-PAAS / TEAM DEMO UI

INGRESS · HTTPS · SERVICE URL

작업이 완료되었습니다.

서비스 접속 정보

k-le-paas-test03-svc의 접속 정보를 찾았습니다

서비스 정보

서비스 이름:

k-le-paas-test03-svc

포트:

80/TCP

네임스페이스:

default

인그리스 정보

도메인 (Host):

k-le-paas-test03.klepaas.app

상태 (Status):

● 활성 (HTTPS)

대상 서비스:

k-le-paas-test03-svc

대상 포트:

80

경로 (Path):

/

보안 리디렉션:

사용 중

서비스 엔드포인트

http://k-le-paas-test03-svc:80

접속 가능한 URL

https://k-le-paas-test03.klepaas.app/

CONTRIBUTED BACKEND FLOW

REQUEST

↓

SERVICE + INGRESS LOOKUP

↓

EXTERNAL HTTPS URL

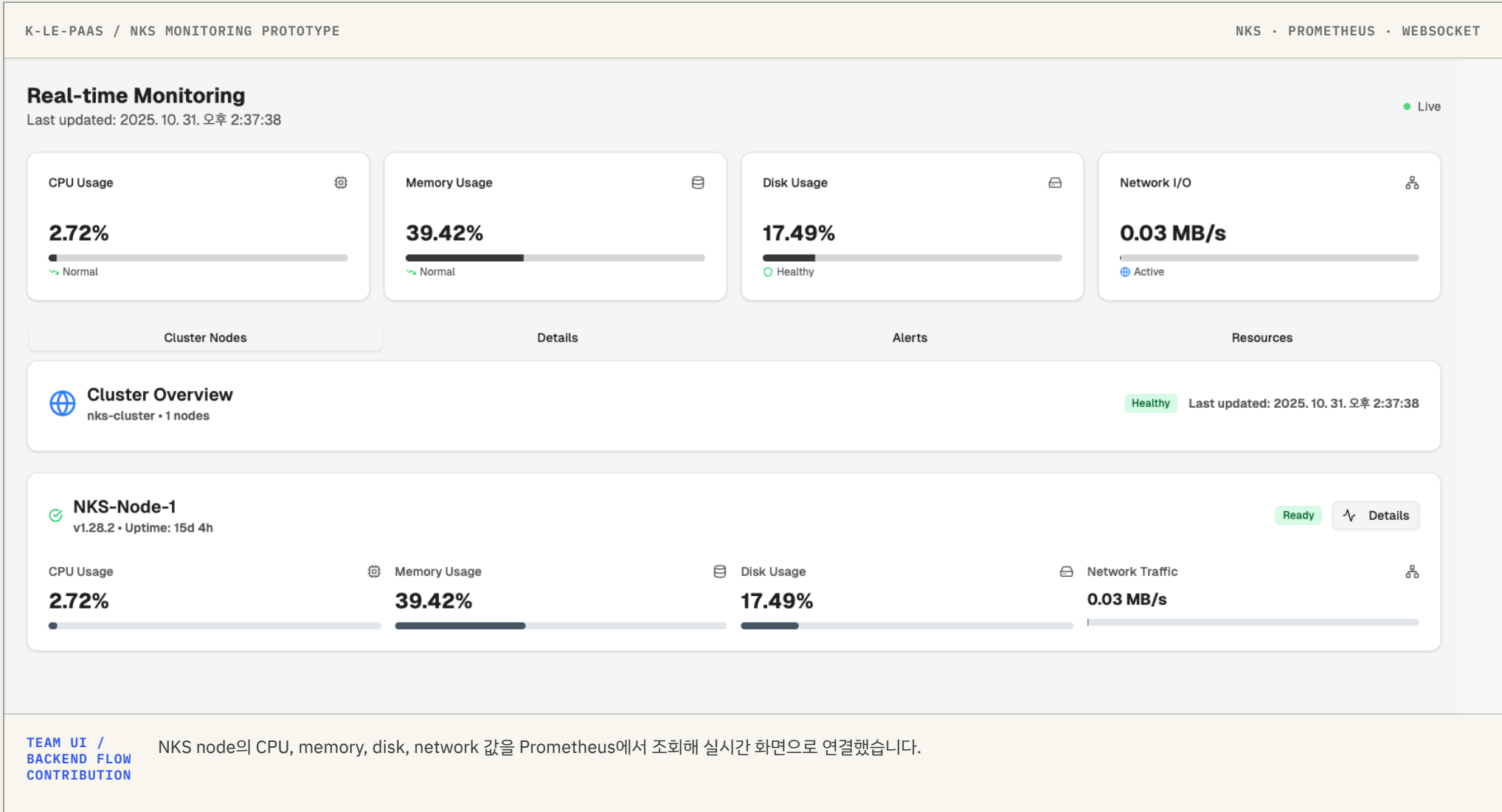
RESULT

서비스 정보와 Ingress host를 HTTPS URL로 구성해 하나의 결과 카드에 표시합니다.

IMPLEMENTATION REFERENCES

직접 구현한 범위를 화면과 PR로 연결했습니다.

팀 제품의 전체 기능과 개인 기여를 분리하고, 대표 구현을 세 개의 PR로 구분했습니다.



MERGED ↗

PR #28

NLP / Kubernetes Operations

자연어 해석 결과를 상태·로그·재시작과 Kubernetes resource 조회에 연결

MERGED ↗

PR #42

NKS Monitoring

Prometheus 기반 CPU·memory·disk·network API와 WebSocket endpoint

MERGED ↗

PR #63

Deployment URL

사용자·저장소별 service URL 생성, 저장, 조회 흐름

GJALLAR

같은 자동화라도
성공 기준은 달랐습니다.

Preview를 활성화하는 판단과 Proxmox의 실제 인프라를 변경하는 판단은 필요한 권한, 실패 방식, 성공을 확인하는 기준이 다릅니다.

DEPLOYMENT CONTROL	HEIMDALL	INFRASTRUCTURE OPERATIONS	GJALLAR
COMMIT → CANDIDATE → HEALTH + ROUTE → ACTIVE		LIVE STATE → PLAN → CONFIRM → POST-CHECK	
성공 기준Preview endpoint가 예상한 응답을 반환		성공 기준API 작업은 task + actual state, guided 작업은 API after-state	
주요 권한repository checkout · Docker runtime · NGINX route		주요 권한RBAC · action-specific acknowledgement · Proxmox API	
실패 처리기존 정상 Preview 유지 · reconciliation		실패 처리non-success 상태 보존 · 자동 mutation retry 금지	

DIFFERENT
TRUST
BOUNDARY

DESIGN RETROSPECTIVE

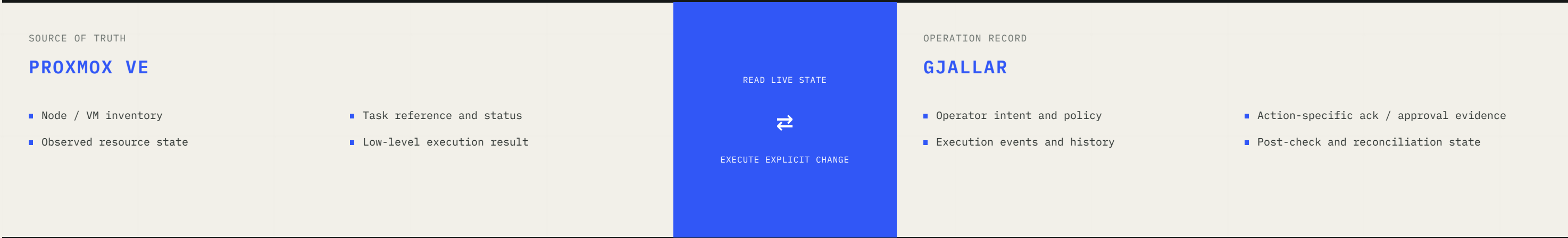
같은 ‘운영 자동화’라도 실제 자원을 바꾸는 권한과 성공 판정은 분리해야 했습니다.

Gjallar는 Proxmox 운영만을 다루는 별도 control surface로 제품 책임을 졌습니다.

OPERATION MODEL

Proxmox는 실제 상태를, Gjallar는 운영 판단을 소유합니다.

운영 도구의 데이터가 실제 인프라 상태를 대신하지 않도록, 기록해야 할 것과 다시 조회해야 할 것을 나눴습니다.



01 MANAGED API	02 GUIDED MANUAL	03 OBSERVE ONLY
Gjallar가 Proxmox API 작업을 실행하고 task와 변경 후 상태를 확인합니다.	짧은 유효시간의 지침만 발급하고, 작업은 운영자가 직접 수행한 뒤 API로 결과를 확인합니다.	capacity와 placement 신호는 조회 시 계산하며 자동 조치나 자동 balancing으로 이어지지 않습니다.

01 OBSERVE	02 PLAN	03 CONFIRM	04 EXECUTE	05 VERIFY	06 RECORD
---------------	------------	---------------	---------------	--------------	--------------

SUCCESS NEEDS EVIDENCE

API 응답을 최종 성공으로 보지 않습니다.

각 작업의 위험과 실행 방식에 맞는 확인 절차를 적용하고, Proxmox가 보여주는
변경 후 상태를 다시 조회합니다.

01CREATE VMMANAGED API	02VM STARTMANAGED API	03GUIDED UNLOCKGUIDED MANUAL
01exact plan + checksum 고정	01stopped, non-template VM 확인	01lock과 active task 사전 확인
02yellow risk acknowledgement / red block	02action acknowledgement	02risk ack + idempotency key
03최종 mutation acknowledgement	03idempotency key로 중복 요청 차단	035분 instruction bundle 발급
04native API dispatch + UPID tracking	04task polling	04운영자가 명령을 직접 수행
05생성 후 VM 상태 확인	05observed state가 running인지 확인	05attestation 뒤 API로 lock 해제 확인

NON-SUCCESS PATH	TIMEOUT 응답 지연	MISSING TASK task reference 없음	POST-CHECK MISMATCH 예상 상태와 불일치	성공으로 바꾸거나 자동 재실행하지 않고 상황에 따라 failed 또는 needs_reconciliation로 남깁니다.
------------------	------------------	-----------------------------------	-----------------------------------	--

프로세스 비정상 종료 뒤의 완전한 자동 복구는 현재 지원하지 않습니다.

NARROW BY DESIGN

현재 지원 범위를 의도적으로 좁게 유지합니다.

지원하지 않는 작업까지 넓게 설명하기보다, 현재 구현하고 자동 테스트한 mutation과 조회 범위를 제품 경계로 명시했습니다.

01 SUPPORTED NOW	02 CONNECTION TRUTH	03 NOT CLAIMED
RBACviewer / operator / admin	UNCONFIGURED ●target 설정 없음	Stop / Shutdown / Reboot / Reset / Delete
LIVE INVENTORYnode / VM / storage / network / task		Snapshot / Rollback
CREATE VMplan, guards, UPID, post-check	LIVE ●실제 Proxmox 조회 가능	Arbitrary shell / SSH / <code>qm</code> executor
GUARDED STARTack, idempotency, observed state		Autonomous remediation
GUIDED UNLOCKmanual execution, API verification	DEGRADED ○연결 실패를 그대로 노출	Automatic balancing
OPERATIONShistory, risk, reconciliation state	Production runtime은 mock 또는 demo inventory로 대체하지 않습니다.	Durable runner / complete restart recovery
INSIGHTSrequest-time, read-only signals		Multi-instance distributed locking

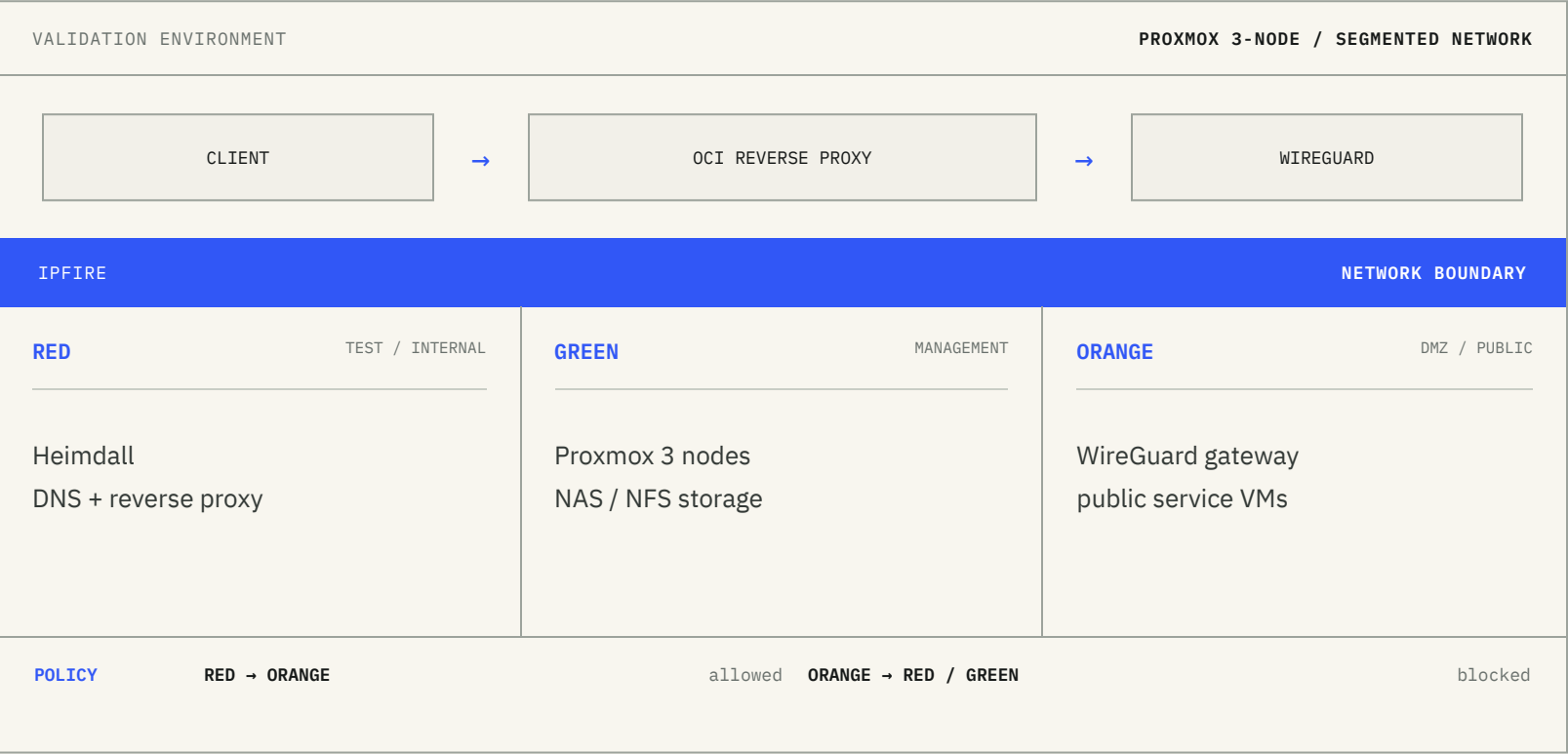
MAINTENANCE SURFACE 기존 DRS route는 read-only 추천·검사와 좁은 migration/reconciliation 호환 경로로 남아 있지만, 현재 제품 정체성은 VM 운영과 공통 operation 기록에 둡니다.

CURRENT CODE VALIDATION BACKEND / 457 TESTS FRONTEND / 17 TESTS · LINT · BUILD PRODUCTION IMAGE BUILD

WHAT I LEARNED

구현만큼 중요했던 건,
실행 결과를 판단하는 기준이었습니다.

01	PIN BEFORE EXECUTION 실행 전에 commit, configuration, plan을 고정합니다.
02	VERIFY THE REAL OUTCOME API 응답과 실제 resource 상태를 분리해서 확인합니다.
03	PRESERVE UNCERTAINTY 확인할 수 없는 상태를 성공으로 단정하지 않습니다.
04	STATE THE BOUNDARY 현재 지원 범위와 다음 단계의 기능을 구분합니다.



VALIDATION SCOPE Heimdall의 single-host 배포와 Gjallar의 이전 Create VM 흐름은 홈랩에서 확인했습니다. Gjallar의 현재 Operations 구조는 자동 테스트와 production build로, K-Le-PaaS는 팀 NKS/NCP 환경에서 구현했습니다.

CLOSING NOTE

배포와 인프라 운영을 자동화하고,
실행 후 실제 상태까지 확인합니다.

더 많은 기능보다 명확한 책임, 안전한 실행 경로, 확인할 수 있는 결과를 만드는 엔지니어입니다.

NAME

조윤호 / YUNHO CHO

ROLE

PLATFORM ENGINEER

WEB

yoonman.page ↗

GITHUB

github.com/CodingPenguin-yoon ↗

EMAIL

code.penguin.yoon@gmail.com ↗

SELECTED REPOSITORIES

HEIMDALL ↗

K-LE-PAAS ↗

GJALLAR ↗